VC Generation

Overview

Crafted by Arie Gurfinkel

Made by Codex 🤖

Is This Program Safe?

```
entry:
  M := havoc
  i := havoc
  x := M[i]
  ok := x == 0
  assert ok
  halt
```

Can `assert ok` fail?

If so: which values of `M` and `i` break it?

Is This Program Safe?

```
entry:
  M := havoc
  i := havoc
  x := M[i]
  ok := x == 0
  assert ok
  halt
```

Can `assert ok` fail?

If so: which values of `M` and `i` break it?

```
$ ttac vcgen unsafe_bytemap.ttac -solve
sat
```

SAT a failing execution exists.

The Counterexample, Replayed

The solver returns a **model** — concrete values for every symbol:

```
$ ttac vcgen unsafe_bytemap.ttac --solve --model m.txt
sat
i = 1    x = 2    ok = false    M = (lambda addr. 2)
```

The Counterexample, Replayed

The solver returns a **model** — concrete values for every symbol:

```
$ ttac vcgen unsafe_bytemap.ttac –solve –model m.txt
sat
i = 1    x = 2    ok = false    M = (lambda addr. 2)
```

The interpreter replays the model into the failing assert:

```
$ ttac run unsafe_bytemap.ttac –model m.txt
status: assert_failed (assertion failed in block entry)
steps: 5    assert_ok: 0    assert_fail: 1
```

What Did The Solver Actually See?

```
(set-logic QF_UFNIA)

(declare-const i Int)
(declare-const x Int)
(declare-const ok Bool)
(declare-const BLK_EXIT Bool)
(declare-fun M (Int) Int)

; block entry
(assert (= x (M i)))
(assert (= ok (= x 0)))

; cfg constraints
(assert (=> BLK_EXIT (not ok)))
(assert BLK_EXIT)

(check-sat)
```

Not the program — this formula: the **verification condition** (VC).

- assignments became equalities
- the assert became its negation
- control flow became constraints

VC generation is the translation. These lectures are about how it works — and how it goes wrong.

The Contract

VC generation turns a TAC-like program into an SMT query:

$$\text{sat}(\text{VC}(P, A)) \Leftrightarrow \text{there exists an execution reaching failure of } \ A$$

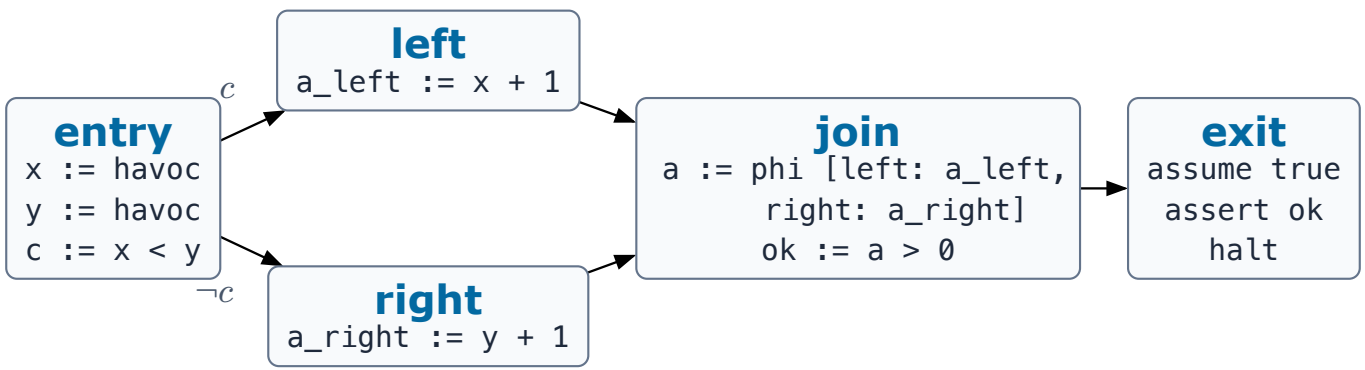
$$\text{SAT} \Rightarrow \text{a bug witness exists — the model is the failing execution}$$

$$\text{UNSAT} \Rightarrow \text{the assertion is proved in the modeled semantics}$$

Tiny TAC (`ttac`) keeps the semantic core visible: basic blocks and terminators, named branch and assert conditions, havoc, assume, maps, and phi nodes at joins.

The Running Example

One diamond carries the whole lecture series — branches, a join, a phi node, definitions, and a final failure query:



Every encoder in this series processes **this** graph.

Three Encodings of the Same Graph

- **SeaHorn-style** — control flow stays explicit: one reachability variable per block, constrained to describe a path.
- **Boogie-style** — control flow becomes recursion: one backward safety equation per block.
- **SeaBMC-style** — control flow disappears: control-dependence is compiled into data-dependence, then the VC is a flat conjunction.

The encodings differ only in how they represent **path semantics** — definitions, assumes, and the failure query look alike in all three.

Series: 01 Tiny TAC · 02 well-formedness · 03 SeaHorn · 04 Boogie · 05 SeaBMC + comparison · 06 references and borrowing